News & Social Media Analytics Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Thota Jhanasri (2520090121)
Sutte Alekhya (2520090126)

Team Number: 02
Section: 12

Under the guidance of

Ms. K. Chandusha
Assistant Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**News & Social Media Analytics Engine**" is a record of Bonafide work of **Thota Jhanasri – 2520090121, Sutte Alekhya – 2520090126** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Thota Jhanasri – 2520090121

Sutte Alekhya – 2520090126

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**News & Social Media Analytics Engine**" is a bonafide work done and submitted by **Thota Jhanasri – 2520090121, Sutte Alekhya – 2520090126** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HODSignature of the External Examiner

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide, Ms. K. Chandusha, Assistant Professor, CS&IT, which cannot be expressed in words, for the tremendous support, encouragement and guidance given for this project.

We express our gratitude to Dr. Khaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Thota Jhanasri – 2520090121

Sutte Alekhya – 2520090126

TABLE OF CONTENTS

1. ABSTRACT6

2. INTRODUCTION7

3. LITERATURE SURVEY8

4. PROBLEM STATEMENT AND OBJECTIVES10

5. HARDWARE AND SOFTWARE REQUIREMENTS11

6. IMPLEMENTATION12

6.1 Phase 1: Console analytics engine12

6.2 Phase 2: Full-stack web application (PulseLens)16

7. RESULTS & DISCUSSION22

7.1 Phase 1 results (console engine)22

7.2 Phase 2 results (web application)23

7.3 Discussion25

8. CONCLUSION & FUTURE SCOPE27

9. REFERENCES28

1. ABSTRACT

The News & Social Media Analytics Engine is a Java-based system that analyses news articles and social-media posts and turns raw text into organised information. With the rapid growth of digital platforms it is difficult to see which topics are popular and which content is most relevant. The project was built in two phases.

Phase 1 (console engine). The engine reads posts from a CSV file, cleans the text by removing special characters and stop words, and performs keyword extraction, hashtag analysis, sentiment classification (positive, negative or neutral) and trending-topic detection. It uses a HashMap for frequency counting, an ArrayList for storing posts, linear and binary search for retrieval, sorting for ranking, and a PriorityQueue (min-heap) for the Top-K keywords, hashtags and trends in O(n log k) time. On a 30-post sample dataset it identified #technology, #sports and #economy as the leading hashtags, classified 46.7% of posts as positive, 13.3% as neutral and 40.0% as negative, and ranked trending topics by combining frequency with likes.

Phase 2 (full-stack web application). The engine was extended into a complete web application named PulseLens. The frontend is built with React and Vite, the backend with Java 21 and Spring Boot 3 exposing REST APIs, and the data is stored in a MySQL 8 database with 16 tables. The application provides registration and login (BCrypt password hashes, session cookies), role-based administration, a dashboard with charts, a news explorer, social-post analytics, trending topics, search history, bookmarks, notifications, text analysis (keywords, a dictionary-based sentiment estimate and an extractive summary), a source-grounded assistant and CSV/print reports. An Advanced DSA Lab runs selected algorithms from all six course outcomes: KMP, Rabin–Karp and Aho–Corasick (string algorithms), Wagner–Fischer edit distance and LCS (dynamic programming), Edmonds–Karp max-flow with min-cut (network flow), a vertex-cover 2-approximation (NP-completeness and approximation), and Miller–Rabin with parallel text analysis (randomised and parallel algorithms).

The results show that choosing the right data structure and algorithm makes the analysis of growing textual data efficient and easy to understand, and that the same algorithms can be delivered as a usable web product.

Team Name: Team 02

Team Logo (if any): —

Team Members:

1. Thota Jhanasri, 2520090121
2. Sutte Alekhya, 2520090126

2. INTRODUCTION

Background. The News & Social Media Analytics Engine is a Java-based application developed to analyse large amounts of news articles and social-media posts in an organised and efficient manner. With the rapid growth of digital platforms, users receive information from many sources, which makes it difficult to identify which topics are popular and which information is most relevant. The proposed system processes textual data and extracts useful information such as frequent keywords, hashtags, trending topics and sentiment patterns.

Motivation and approach. The main focus of the project is the practical application of Data Structures and Algorithms to a real-world analytics problem. A HashMap is used for efficient frequency counting, Lists for storing and managing data, searching and sorting algorithms for retrieving and ranking information, and a Priority Queue for finding the most important items. The first version works from CSV/TXT files and a 30-post sample dataset so that every result can be checked by hand.

Extension to a full-stack system. To make the engine usable by non-programmers and to apply the advanced algorithm families studied in this course (string algorithms, advanced dynamic programming, network flow, NP-completeness and approximation, randomised and parallel algorithms), the engine was extended into a web application. A React frontend talks to a Spring Boot REST backend, which stores users, articles, posts, search history, bookmarks and analysis results in MySQL. Each advanced algorithm is available both as part of a feature and as an interactive lab page where the user supplies their own input and sees the result and execution time.

Organisation of the report. Section 3 reviews related work; Section 4 states the problem and objectives; Section 5 lists the hardware and software used; Section 6 presents the implementation of Phase 1 (console engine) and Phase 2 (full-stack application); Section 7 shows the results with discussion; and Section 8 concludes with future scope, followed by the references.

3. LITERATURE SURVEY

1. Apache Storm – A distributed real-time stream-processing framework, written for the Java Virtual Machine, in which spouts and bolts process unbounded streams of data. It is widely used for live analytics such as counting hashtags in a stream. Our project applies the same frequency-counting idea on a single machine using a HashMap.

2. Apache Kafka – A distributed log used to ingest and replay high-volume event streams such as posts, clicks and logs. It decouples the producers of data from the analytics programs that consume it. In our project the same role is played, on a small scale, by the CSV file in Phase 1 and by the MySQL tables in Phase 2.

3. Elasticsearch / Apache Lucene – A search engine, written in Java, that builds an inverted index mapping every term to the documents containing it. It supports fast full-text and keyword queries over very large collections. Our engine offers keyword search through linear search over posts, binary search over a sorted keyword list and, in Phase 2, KMP, Rabin–Karp and Aho–Corasick pattern matching.

4. NetworkX – A Python library for analysing graphs and social networks, including shortest paths and PageRank. It is commonly used to find influential users in a network. Our Phase 2 lab applies graph algorithms (maximum flow, minimum cut and vertex cover) to user-supplied graphs, and the schema includes a citation_edges table for relationships between articles.

5. VADER Sentiment Analysis – A lexicon and rule-based sentiment tool designed for social-media text, introduced by Hutto and Gilbert. Each word carries a polarity score and the scores are combined to label a post. Our classifier follows the same lexicon principle with small positive and negative word sets.

6. GDELT Project – A large open database that monitors news media worldwide and extracts themes, people, locations and tone from articles. It shows the value of analysing news at scale and over time. Our project applies a much smaller version of the same idea to keywords, hashtags and sentiment.

7. Google Trends – A public service that reports how often topics are searched and which topics are rising. Our engine provides a comparable feature by ranking topics with a score that combines frequency and likes, and the web application lists frequent terms as trending topics.

8. Brandwatch – A commercial social-listening platform that tracks mentions of brands and topics across social networks and news sites, with sentiment reports. It shows the practical demand for such tools, but its algorithms are closed. Our engine reproduces the main features openly so that each step can be studied.

9. Sprout Social – A commercial social-media management suite that provides engagement analytics such as likes and shares per post and highlights the best-performing content. We use

a similar idea to rank posts by likes through sorting and to compute total engagement (likes + shares + comments) in the social page.

Comparison of existing approaches. The table below compares the kinds of existing systems with the proposed system.

S.No	Existing approach	Technology / method	Limitations
1	News aggregation systems	Web APIs, web scraping	Mainly collect and display news
2	Keyword analysis systems	Hashing, frequency counting	Limited trend analysis
3	Sentiment analysis systems	Text processing	Often focus only on sentiment
4	Trending topic systems	Sorting, ranking	May not combine multiple analytics
5	Social media analytics	Hashtag and keyword analysis	Can be difficult to process large datasets
6	Proposed system	Java + DSA, Spring Boot, React, MySQL	Combines keyword, hashtag, sentiment and trend analysis with an interactive advanced-algorithm lab

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: To design a Java application that applies Data Structures and Algorithms to analyse news and social-media posts and report the popular keywords, hashtags, sentiment and trending topics, and to deliver the analysis through a secure full-stack web application.

Problem Description: Users receive far more information than they can read, and it is hard to tell which topics are popular. Efficient data structures and algorithms are needed to count, search, match and rank this text quickly, and a usable interface with accounts, storage and reports is needed so that the results can be explored and kept.

Objectives:

Purpose of the project: to solve the problem of information overload in news and social media by building each analysis feature on a suitable data structure or algorithm. The objectives are:

- To read news and social-media data from CSV/TXT files and store it in an ArrayList of Post objects (Phase 1).
- To clean the text by removing special characters and stop words (HashSet look-up).
- To count keywords and hashtags efficiently using a HashMap (O(1) average per update).
- To classify every post as Positive, Negative or Neutral using word lists.
- To find the Top-K keywords, hashtags and trending topics using a PriorityQueue (min-heap) in O(n log k).
- To rank posts by likes using sorting and to retrieve information using linear and binary search.
- To build a Spring Boot REST backend and a MySQL database that store users, articles, posts, bookmarks, search history, notifications and analysis results (Phase 2).
- To provide secure registration, login and role-based administration using BCrypt password hashing and session authentication.
- To build a responsive React dashboard with charts, a news explorer, social analytics, text analysis, reports and a dark/light theme.
- To implement and demonstrate selected algorithms from all six course outcomes in an Advanced DSA Lab: KMP, Rabin–Karp, Aho–Corasick, edit distance, LCS, Edmonds–Karp, vertex-cover approximation, Miller–Rabin and parallel text analysis.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 or above – runs the application (i5 recommended for running MySQL, Spring Boot and Vite together)
- RAM: 4 GB minimum for Phase 1; 8 GB recommended for Phase 2
- Storage: 10 GB free space – stores project files, dependencies and the database
- Monitor: 14" or above – displays the application
- Keyboard and Mouse: standard – user interaction

Software Requirements:

Software	Version / tool	Purpose
Java	JDK 17 (Phase 1), JDK 21 (Phase 2)	Main programming language
Spring Boot	3.5.5 (Web, Data JPA, Security, Validation)	REST API, persistence, authentication
Maven	3.9 or later	Build and dependency management
MySQL	8 or later, with MySQL Workbench	Relational database
Node.js	20 or later	Runs the frontend toolchain
React and Vite	React 18.3, Vite 6	Frontend library and dev server
Frontend libraries	React Router 6, Axios, Recharts, lucide-react	Routing, HTTP calls, charts, icons
IDE	IntelliJ IDEA / Eclipse / VS Code	Development environment
Java Collections Framework	java.util	Implements data structures and algorithms
CSV / TXT files	—	Phase 1 data input
Git / GitHub	—	Version control and code management
Web browser	Chrome, Edge or Firefox	Runs the web application

6. IMPLEMENTATION

6.1 Phase 1: Console analytics engine

Methodology. The workflow in Figure 6.1 is followed. Data is read from a TXT/CSV file, pre-processed (stop words and special characters removed) and analysed in four parallel ways – keyword extraction, hashtag analysis, sentiment analysis and trend detection – after which the results are stored and displayed. The system is implemented in four modules whose complete source code follows.

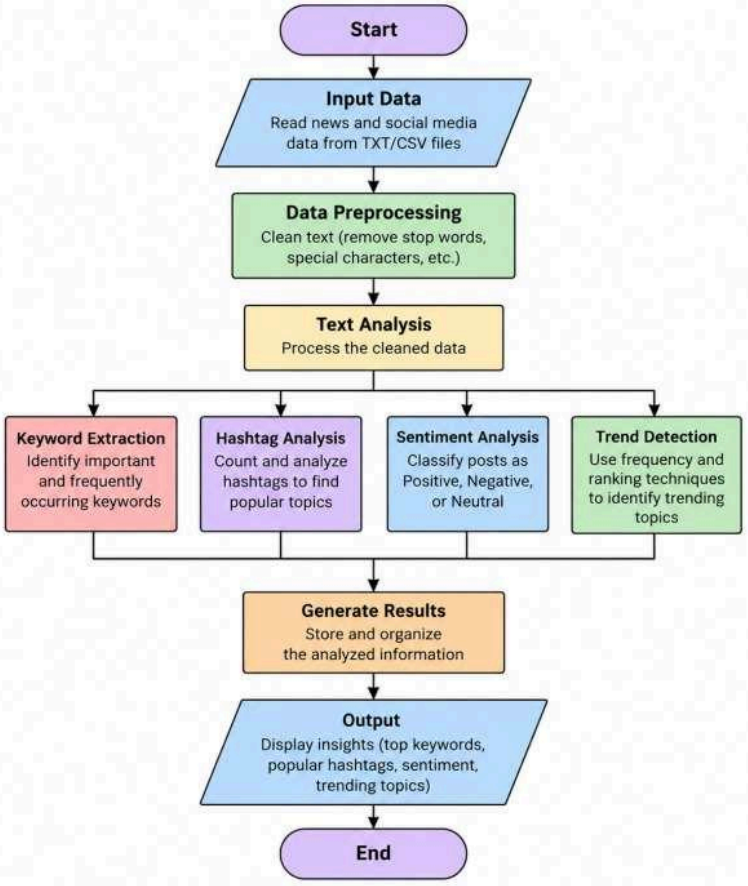


Figure 6.1: Methodology / workflow of the engine

Module 1: Post.java and DataLoader.java – reading the CSV data

```
/** A single news article or social-media post read from the CSV file. */
public class Post {
    final int id;
    final String source;   // "News" or "Social"
    final int likes;
    final String text;
    String sentiment;      // Positive / Negative / Neutral (filled by Analyzer)

    Post(int id, String source, int likes, String text) {
        this.id = id; this.source = source; this.likes = likes; this.text = text;
    }
    public String toString() {
        return "#" + id + " [" + source + "] likes=" + likes + " | " + text;
    }
}

import java.io.*;
import java.util.*;
/** Module 1: reads the CSV / TXT data file into an ArrayList of Post objects. */
public class DataLoader {
```

```
    public static List<Post> load(String path) throws IOException {
        List<Post> posts = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            String line = br.readLine();               // skip header row
            while ((line = br.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] p = line.split(",", 4);       // id,source,likes,text
                posts.add(new Post(Integer.parseInt(p[0]), p[1],
                                   Integer.parseInt(p[2]), p[3]));
            }
        }
        return posts;
    }
}
```

Module 2: TextPreprocessor.java – cleaning the text

```
import java.util.*;
/** Module 2: cleans text - lower-case, removes special characters and stop words. */
public class TextPreprocessor {
    private static final Set<String> STOP = new HashSet<>(Arrays.asList(

"the","a","an","is","are","was","to","of","and","in","on","for","as","at","by","it",
        "this","that","with","from","after","again","new","our","we","if","do","not","but",

"so","be","has","have","all","every","during","ahead","today","week","year","across"));

    /** Extracts hashtags such as #cricket before special characters are removed. */
    public static List<String> hashtags(String text) {
        List<String> tags = new ArrayList<>();
        for (String w : text.split("\\s+"))
            if (w.startsWith("#") && w.length() > 1) tags.add(w.toLowerCase());
        return tags;
    }
    /** Returns the cleaned list of meaningful words (no hashtags, no stop words). */
    public static List<String> clean(String text) {
        List<String> words = new ArrayList<>();
        for (String w : text.split("\\s+")) {
            if (w.startsWith("#")) continue;
            w = w.toLowerCase().replaceAll("[^a-z]", "");   // strip special characters
            if (w.length() > 2 && !STOP.contains(w)) words.add(w);
        }
        return words;
    }
}
```

Module 3: Analyzer.java – keyword, hashtag, sentiment and trend analysis, sorting and searching

```
import java.util.*;
/** Module 3: keyword, hashtag, sentiment and trend analysis plus searching / sorting. */
public class Analyzer {
    private static final Set<String> POS = new HashSet<>(Arrays.asList(
        "great","amazing","excellent","happy","win","success","love","loved","good","best",
        "proud","congratulations","boost","growth","record","better","faster","strong"));
    private static final Set<String> NEG = new HashSet<>(Arrays.asList(
        "bad","terrible","worst","angry","sad","loss","crisis","risk","failed","attack",
        "crashed","delayed","disappointed","burden","outage","flood"));

    final List<Post> posts;
    final Map<String,Integer> keywordFreq = new HashMap<>();   // HashMap: frequency
counting
    final Map<String,Integer> hashtagFreq = new HashMap<>();
    final Map<String,Double> trendScore = new HashMap<>();
    final Map<String,Integer> sentimentCount = new LinkedHashMap<>();

    Analyzer(List<Post> posts) {
        this.posts = posts;
        sentimentCount.put("Positive", 0); sentimentCount.put("Neutral", 0);
        sentimentCount.put("Negative", 0);
        for (Post p : posts) {
            List<String> words = TextPreprocessor.clean(p.text);
            for (String w : words) {
                keywordFreq.merge(w, 1, Integer::sum);
                trendScore.merge(w, 1 + p.likes / 100.0, Double::sum);   //
engagement-weighted
            }
            for (String t : TextPreprocessor.hashtags(p.text)) {
                hashtagFreq.merge(t, 1, Integer::sum);
                trendScore.merge(t, 1 + p.likes / 100.0, Double::sum);
```

```
            }
            p.sentiment = classify(words);
            sentimentCount.merge(p.sentiment, 1, Integer::sum);
        }
    }
    private String classify(List<String> words) {
        int s = 0;
        for (String w : words) { if (POS.contains(w)) s++; if (NEG.contains(w)) s--; }
        return s > 0 ? "Positive" : s < 0 ? "Negative" : "Neutral";
    }
    /** Top-K entries using a PriorityQueue (min-heap of size k): O(n log k). */
    static <T extends Number> List<Map.Entry<String,T>> topK(Map<String,T> map, int k) {
        PriorityQueue<Map.Entry<String,T>> pq = new PriorityQueue<>(
            (a, b) -> Double.compare(a.getValue().doubleValue(),
b.getValue().doubleValue()));
        for (Map.Entry<String,T> e : map.entrySet()) {
            pq.offer(e);
            if (pq.size() > k) pq.poll();            // drop the smallest
        }
        List<Map.Entry<String,T>> out = new ArrayList<>(pq);
        out.sort((a, b) -> Double.compare(b.getValue().doubleValue(),
a.getValue().doubleValue()));
        return out;
    }
    /** Sorting: posts ranked by likes, highest first. */
    List<Post> rankByLikes() {
        List<Post> sorted = new ArrayList<>(posts);
        sorted.sort((a, b) -> b.likes - a.likes);
        return sorted;
    }
    /** Linear search: all posts that contain the keyword. */
    List<Post> linearSearch(String keyword) {
        List<Post> found = new ArrayList<>();
        for (Post p : posts)
            if (p.text.toLowerCase().contains(keyword.toLowerCase())) found.add(p);
        return found;
    }
    /** Binary search on the alphabetically sorted keyword list: O(log n). */
    int binarySearch(List<String> sortedWords, String key) {
        int lo = 0, hi = sortedWords.size() - 1;
        while (lo <= hi) {
            int mid = (lo + hi) >>> 1, c = sortedWords.get(mid).compareTo(key);
            if (c == 0) return mid;
            if (c < 0) lo = mid + 1; else hi = mid - 1;
        }
        return -1;
    }
}
```

Module 4: Main.java – driver program

```
import java.util.*;
/** Module 4: driver - runs the full pipeline and prints the output. */
public class Main {
    public static void main(String[] args) throws Exception {
        List<Post> posts = DataLoader.load("posts.csv");
        Analyzer an = new Analyzer(posts);
        int news = 0; for (Post p : posts) if (p.source.equals("News")) news++;
        System.out.println("Posts loaded: " + posts.size() + " (News: " + news
                           + ", Social: " + (posts.size() - news) + ")");
        System.out.println("Distinct keywords: " + an.keywordFreq.size()
                           + "  Distinct hashtags: " + an.hashtagFreq.size());

        System.out.println("\n[1] TOP 5 KEYWORDS (HashMap + PriorityQueue)");
        int i = 1; for (Map.Entry<String,Integer> e : Analyzer.topK(an.keywordFreq, 5))
            System.out.printf("  %d. %-14s %d times%n", i++, e.getKey(), e.getValue());

        System.out.println("\n[2] TOP 5 HASHTAGS (HashMap + PriorityQueue)");
        i = 1; for (Map.Entry<String,Integer> e : Analyzer.topK(an.hashtagFreq, 5))
            System.out.printf("  %d. %-14s %d times%n", i++, e.getKey(), e.getValue());

        System.out.println("\n[3] SENTIMENT ANALYSIS");
        for (Map.Entry<String,Integer> e : an.sentimentCount.entrySet())
            System.out.printf("  %-9s %2d posts (%.1f%%)%n", e.getKey(), e.getValue(),
                              100.0 * e.getValue() / posts.size());

        System.out.println("\n[4] TRENDING TOPICS (frequency + likes, PriorityQueue
ranking)");
        i = 1; for (Map.Entry<String,Double> e : Analyzer.topK(an.trendScore, 5))
```

```
            System.out.printf("  %d. %-14s trend score %.2f%n", i++, e.getKey(),
e.getValue());

        System.out.println("\n[5] TOP 3 POSTS BY LIKES (Sorting)");
        List<Post> ranked = an.rankByLikes();
        for (int k = 0; k < 3; k++) System.out.println("  " + ranked.get(k));

        System.out.println("\n[6] SEARCH");
        List<Post> found = an.linearSearch("cricket");
        System.out.println("  Linear search 'cricket' -> " + found.size() + " posts: ids "
+ ids(found));
        List<String> sortedWords = new ArrayList<>(an.keywordFreq.keySet());
        Collections.sort(sortedWords);
        System.out.println("  Binary search 'technology' -> index "
                           + an.binarySearch(sortedWords, "technology") + " of " +
sortedWords.size());
        System.out.println("  Binary search 'zebra' -> index "
                           + an.binarySearch(sortedWords, "zebra") + " (not found)");
    }
    static String ids(List<Post> l) {
        List<Integer> r = new ArrayList<>(); for (Post p : l) r.add(p.id); return
r.toString();
    }
}
```

6.2 Phase 2: Full-stack web application (PulseLens)

Phase 2 turns the console engine into a web application. The user interface is a React single-page application, the server is a Spring Boot REST service, and all data is stored in MySQL. Selected advanced algorithms from the six course outcomes are integrated into the analysis features and into an interactive lab.

6.2.1 System architecture

The system follows a three-tier architecture. The browser runs the React application on port 5173 and calls the backend through JSON over HTTP using Axios. The Spring Boot service on port 8080 validates requests, applies security rules, runs the analysis and algorithm code, and reads or writes the MySQL database through Spring Data JPA (Hibernate). Cross-origin requests are allowed only from the frontend origin.

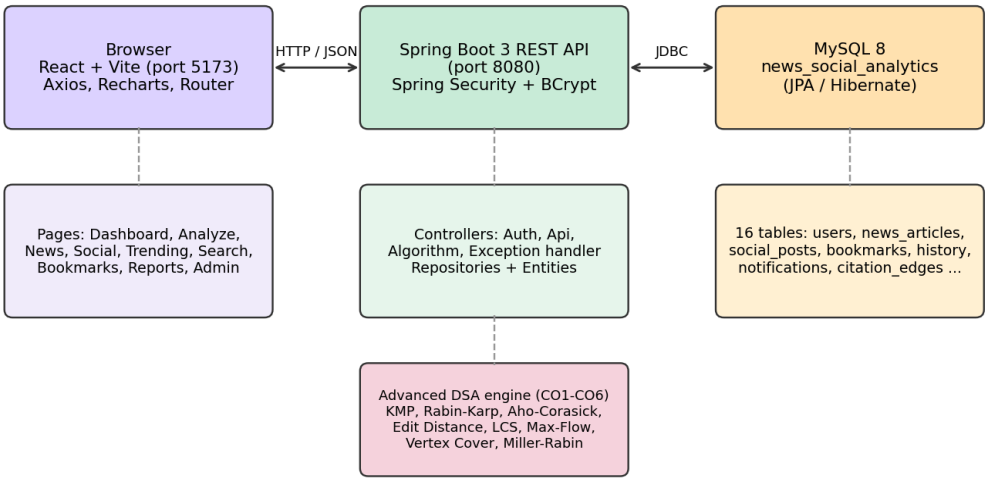


Figure 6.2: Three-tier architecture of the full-stack system

6.2.2 Technology stack

Layer	Technology	Role
Frontend	React 18, Vite 6, React Router, Axios, Recharts, lucide-react	Pages, routing, charts, API calls, light and dark themes
Backend	Java 21, Spring Boot 3.5.5, Spring Web, Spring Data JPA, Spring Security, Bean Validation	REST endpoints, persistence, authentication, input validation
Database	MySQL 8, schema.sql and seed.sql	Users, articles, posts, history, notifications, results
Security	BCrypt password encoder, HTTP session cookie	Password hashing and session authentication
Build and run	Maven, npm, start-backend.bat, start-frontend.bat	Build and one-click start on Windows

6.2.3 Database design

The file database/schema.sql creates the database tables with primary keys, unique constraints, foreign keys with cascading rules and indexes on frequently searched columns (publication date, category, user and date). The file database/seed.sql inserts clearly labelled sample articles and social posts so that the dashboard is not empty on first run. Seven tables are mapped as JPA entities and used directly by the API (users, news_articles, social_posts, bookmarks, search_history, analysis_history and notifications); the remaining tables are defined for the planned extensions.

Table	Main columns	Purpose
users	id, full_name, username, email, password_hash, role, account_enabled	Registered accounts and roles (USER or ADMIN)
news_articles	id, title, description, content, source_name, category_name, published_at, sentiment	Stored news records with indexes on date and category
social_posts	id, platform, author_name, post_text, likes_count, shares_count, comments_count, hashtag_text, sentiment	Stored social-media posts and engagement counts
bookmarks	user_id, article_id (unique pair)	Reading list of each user
search_history	user_id, query_text, result_count, searched_at	Recent searches of each user
analysis_history	user_id, analysis_type, input_preview, result_json, execution_ms	Saved results of the Analyze feature
notifications	user_id, title, message, read_flag	User alerts
categories, news_sources	name, description, reliability_score	Reference data for categories and sources
hashtags, post_hashtags	tag; post_id, hashtag_id	Many-to-many link between posts and hashtags
user_preferences	theme, preferred_categories, notifications_enabled	Per-user settings
sentiment_results, trending_topics	label and scores; topic, mention_count, period	Stored sentiment and topic results

Table	Main columns	Purpose
algorithm_runs	algorithm_name, input_preview, output_json, execution_ms	Log of algorithm lab runs
citation_edges	from_article_id, to_article_id, relationship_type, weight	Graph of related articles for network algorithms

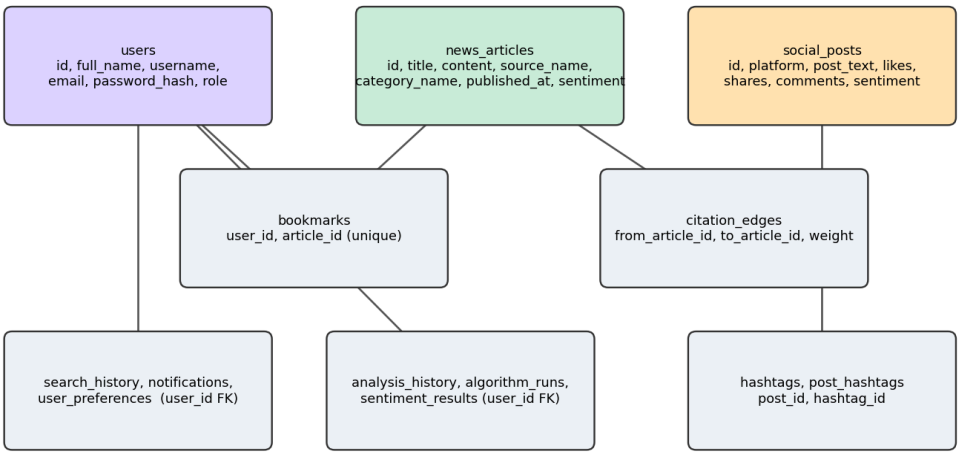


Figure 6.3: Entity-relationship overview of the MySQL database

6.2.4 Backend REST API

The backend is organised into entities, repositories, controllers, configuration and security packages. Request bodies are validated with Bean Validation, and a global exception handler returns clear JSON error messages. The main endpoints are listed below.

Access	Endpoint	Purpose
Public	GET /api/health	Service status
Public	POST /api/auth/register, POST /api/auth/login	Create an account and sign in
Public	GET /api/news, /api/news/{id}, /api/social, /api/social/summary, /api/trending	Browse and filter articles, posts and trending topics
Signed in	GET /api/auth/me, POST /api/auth/logout	Current user and sign-out
Signed in	GET /api/dashboard	Counts, sentiment totals and activity for the charts
Signed in	POST /api/analysis	Keyword, sentiment, summary analysis of supplied text
Signed in	GET/POST /api/search-history, GET /api/analysis-history	Search and analysis history
Signed in	GET/POST/DELETE /api/bookmarks	Reading list
Signed in	GET /api/notifications, POST /api/notifications/{id}/read	Notifications
Signed in	POST /api/assistant	Question answered from stored articles, with sources

Access	Endpoint	Purpose
Signed in	POST /api/algorithms/kmp, rabin-karp, aho-corasick, edit-distance, lcs, max-flow, vertex-cover, miller-rabin, parallel-analysis	Advanced DSA lab
Admin only	GET /api/admin/users, POST/PUT/DELETE /api/admin/news	Manage users and article records

6.2.5 Security design

- Passwords are never stored in plain text: they are hashed with BCrypt before saving, and registration requires at least 8 characters with upper-case, lower-case, a digit and a symbol.
- Login creates a server-side session; the session cookie is HttpOnly with SameSite=Lax and expires after 45 minutes.
- Spring Security allows only the public endpoints without login, requires authentication for all other API calls, and restricts /api/admin/** to the ADMIN role.
- Cross-origin requests are accepted only from the configured frontend origin (http://localhost:5173).
- Inputs are validated on the server, and uploads in the Analyze page are limited to 3 MB of text or CSV.
- Database credentials are read from the environment variables DB_USERNAME and DB_PASSWORD instead of being hard-coded.

6.2.6 Frontend design

The frontend is a responsive single-page application with a sidebar, a top bar with global search, a light lavender theme and a dark theme. Pages that need data show a helpful empty state when no records exist, and API errors are shown in a banner.

Page	What it does
Login / Register	Account creation and sign-in with validation messages
Dashboard	Metric cards, content-activity area chart, sentiment donut chart, latest articles and trending topics
Analyze anything	Paste text or upload .txt/.csv; choose keyword frequency, sentiment estimate, extractive summary; export JSON
News explorer	Search and category filter over stored articles; bookmark an article
Social analytics	Posts with likes, shares and hashtags; totals of engagement and platforms
Trending topics	Most frequent terms across articles and posts
Advanced search and history	Search articles and review recent searches
AI insights and assistant	Source-grounded answers from stored articles and quick text insights
Reports	Export metrics as CSV or print the page to PDF

Page	What it does
Bookmarks, Notifications, Profile, Settings	Personal reading list, alerts, account details and theme choice
Advanced DSA Lab (CO1 to CO6)	Run each algorithm with your own input and see the result and execution time
Admin panel	Add article records and list registered users (ADMIN role only)

6.2.7 Text analysis module

The Analyze feature combines three lightweight methods in one request. Keywords are found by splitting the text into words, ignoring stop words and counting with a HashMap. Sentiment is estimated from positive and negative word lists and reported as positive, neutral and negative scores. The summary is extractive: it selects the most informative sentences from the input. The trending list uses the same frequency counting over stored articles and posts. These methods are intentionally simple and are labelled as lightweight estimates, not trained machine-learning models. The assistant page searches the stored article records and returns matching excerpts with their sources; it does not call an external language model.

6.2.8 Advanced DSA lab: mapping to the six course outcomes

All six course outcomes are covered by selected algorithms. Not every topic of every module is implemented; the algorithms chosen are those that fit the news and social-media analytics problem.

CO	Module topic	Algorithm implemented	Use in the application	Complexity
CO1	TextHack as a system	Central analysis workspace; each query class is mapped to an algorithm family	Analyze page and Lab (CO1) dispatch the request to pattern search or keyword analysis	See Table 7.3
CO2	String algorithms	KMP; Rabin–Karp with a rolling hash; Aho–Corasick (trie with failure links)	Keyword and multi-keyword search in article text	O(n+m); expected O(n+m); O(n+m+z)
CO3	Advanced dynamic programming	Wagner–Fischer edit distance; Longest Common Subsequence	Fuzzy matching of words and similarity of two texts	O(nm)
CO4	Network flow	Edmonds–Karp maximum flow with source-side minimum cut	Capacity-constrained flow on a user-supplied graph	$O(V \cdot E^2)$
CO5	NP-completeness and approximation	Vertex-cover 2-approximation using a maximal matching	Choosing a small set of nodes that covers every relationship	Linear in edges; size $\leq 2 \times$ optimal
CO6	Randomised and parallel algorithms	Miller–Rabin primality test; parallel independent text analysis	Prime testing of large integers; analysing many texts at once with the JVM ForkJoin pool	$O(k \log^3 n)$; depends on workload

Selected code from the algorithm controller is shown below. The Wagner–Fischer table is filled row by row, the Edmonds–Karp loop repeatedly finds a shortest augmenting path by breadth-first search in the residual graph, and the vertex-cover routine picks both endpoints of every edge that is not yet covered.

Edit distance (CO3)

```
int[][] d = new int[a.length()+1][b.length()+1];
for (int i = 0; i <= a.length(); i++) d[i][0] = i;
for (int j = 0; j <= b.length(); j++) d[0][j] = j;
for (int i = 1; i <= a.length(); i++)
    for (int j = 1; j <= b.length(); j++)
        d[i][j] = Math.min(Math.min(d[i-1][j] + 1, d[i][j-1] + 1),
                           d[i-1][j-1] + (a.charAt(i-1) == b.charAt(j-1) ? 0 : 1));
// d[a.length()][b.length()] is the edit distance
```

Edmonds–Karp maximum flow and minimum cut (CO4)

```
long[][] res = new long[n][n];                    // residual capacities
for (int i = 0; i < n; i++) res[i] = cap[i].clone();
long flow = 0;
int[] parent = new int[n];
while (bfs(res, source, sink, parent)) {          // shortest augmenting path
    long aug = Long.MAX_VALUE;
    for (int v = sink; v != source; v = parent[v]) aug = Math.min(aug, res[parent[v]][v]);
    for (int v = sink; v != source; v = parent[v]) {
        int u = parent[v];
        res[u][v] -= aug;  res[v][u] += aug;      // update residual graph
    }
    flow += aug;
}
boolean[] seen = new boolean[n];
reach(res, source, seen);                          // source side of the minimum cut
```

Vertex-cover 2-approximation (CO5)

```
Set<Integer> selected = new TreeSet<>();
for (List<Integer> e : edges) {
    int u = e.get(0), v = e.get(1);
    if (selected.contains(u) || selected.contains(v)) continue;  // edge already covered
    selected.add(u); selected.add(v);                            // take both endpoints
}
// the chosen edges form a maximal matching, so |selected| <= 2 * optimal
```

6.2.9 Project structure and how to run

```
News-Social-Media-Analytics-Engine/
  backend/    pom.xml, src/main/java/com/pulselens/analytics/
              (controller, entity, repository, security, config)
  frontend/   package.json, vite.config.js, src/App.jsx, styles.css, api.js
  database/   schema.sql, seed.sql
  docs/       API.md, CO_MAPPING.md
  start-backend.bat, start-frontend.bat, README.md
```

1. Install JDK 21, Maven 3.9, Node.js 20 and MySQL 8.

2. In MySQL Workbench run database/schema.sql and then database/seed.sql.

3. Set the DB_USERNAME and DB_PASSWORD environment variables, or edit backend/src/main/resources/application.properties.

4. Backend: open a terminal in the backend folder and run mvn spring-boot:run (API at http://localhost:8080).

5. Frontend: open a second terminal in the frontend folder and run npm install and then npm run dev (website at http://localhost:5173).

6. Register an account. To make it an administrator run UPDATE users SET role='ADMIN' WHERE email='your-email'; and sign in again.

7. RESULTS & DISCUSSION

7.1 Phase 1 results (console engine)

The program was compiled with javac and run with java Main on the 30-post sample file posts.csv. The console output is shown in three screenshots.

```
java Main

============================================================
   NEWS & SOCIAL MEDIA ANALYTICS ENGINE  (Java + DSA)
============================================================
Posts loaded: 30  (News: 13, Social: 17)
Distinct keywords: 148   Distinct hashtags: 31

[1] TOP 5 KEYWORDS (HashMap + PriorityQueue)
   1. win            3 times
   2. record         3 times
   3. final          3 times
   4. release        2 times
   5. growth         2 times

[2] TOP 5 HASHTAGS (HashMap + PriorityQueue)
   1. #technology    7 times
   2. #sports        6 times
   3. #economy       5 times
   4. #flood         3 times
   5. #weather       3 times
```

Figure 7.1: Data summary, Top-5 keywords and Top-5 hashtags

The engine loaded 30 posts (13 news, 17 social) and found 148 distinct keywords and 31 hashtags. HashMap counts and a PriorityQueue give #technology (7), #sports (6) and #economy (5) as the most used hashtags.

```
java Main

[3] SENTIMENT ANALYSIS
   Positive  14 posts (46.7%)
   Neutral    4 posts (13.3%)
   Negative  12 posts (40.0%)

[4] TRENDING TOPICS (frequency + likes, PriorityQueue ranking)
   1. #sports        trend score 23.35
   2. #technology    trend score 22.30
   3. win            trend score 13.90
   4. #economy       trend score 12.50
   5. final          trend score 12.15
```

Figure 7.2: Sentiment analysis and trending topics

46.7% of posts were classified positive, 13.3% neutral and 40.0% negative. Trending topics combine frequency with likes, so #sports (23.35) and #technology (22.30) rank highest.

```
java Main

[5] TOP 3 POSTS BY LIKES (Sorting)
   #6 [Social] likes=510 | Amazing performance by the team in the cricket final #cricket #sports
   #25 [Social] likes=480 | AI is amazing but there is risk if we do not regulate it #AI #technology
   #13 [Social] likes=420 | Great news the startup raised big funding to boost growth #startup #economy

[6] SEARCH
   Linear search 'cricket' -> 2 posts: ids [2, 6]
   Binary search 'technology' -> index 129 of 148
   Binary search 'football'   -> index 48 of 148
   Binary search 'zebra'      -> index -1 (not found)
```

Figure 7.3: Top posts by likes (sorting) and searching

Sorting ranks post #6 (510 likes) first. Linear search finds the keyword “cricket” in posts 2 and 6, and binary search locates “technology” and “football” in the sorted keyword list and correctly reports -1 for “zebra”.

7.2 Phase 2 results (web application)

The web application is run as described in Section 6.2.9. The screenshots below show the main pages; they should be captured from the running application and pasted into the placeholders.

[Insert screenshot: Login and registration page]

Figure 7.4: Login and registration page

[Insert screenshot: Dashboard with metrics, activity chart and sentiment donut]

Figure 7.5: Dashboard with metrics, activity chart and sentiment donut

[Insert screenshot: News explorer with search and category filter]

Figure 7.6: News explorer with search and category filter

[Insert screenshot: Analyze anything – keywords, sentiment and summary]

Figure 7.7: Analyze anything – keywords, sentiment and summary

[Insert screenshot: Advanced DSA Lab – algorithm result with execution time]

Figure 7.8: Advanced DSA Lab – algorithm result with execution time

[Insert screenshot: Admin panel – add article and registered users]

Figure 7.9: Admin panel – add article and registered users

Algorithm lab checks. Table 7.1 lists test inputs for the lab with the result expected from a hand calculation. They can be entered on the Advanced DSA Lab pages and the outputs compared with this table.

CO	Algorithm	Input	Expected result (by hand)
CO2	KMP	Text “ababcabcabababd”, pattern “ababd”	Match at position 10, count 1
CO2	Rabin–Karp	Same text and pattern	Match at position 10, count 1
CO2	Aho–Corasick	Text “ushers”; patterns he, she, his, hers	she at 1; he at 2; hers at 2; his not found
CO3	Edit distance	“kitten” and “sitting”	Distance 3

CO	Algorithm	Input	Expected result (by hand)
CO3	LCS	“ABCBDAB” and “BDCABA”	Length 4
CO4	Edmonds–Karp	4 vertices; edges (0,1,3) (0,2,2) (1,2,1) (1,3,2) (2,3,4); source 0, sink 3	Maximum flow 5; source side of the minimum cut {0}
CO5	Vertex cover	4 vertices; edges (0,1) (1,2) (2,3) (0,3)	Selected {0,1,2,3}, size 4; optimal is 2, so the ratio is exactly 2
CO6	Miller–Rabin	561 (a Carmichael number) and 1000000007	561 is composite; 1000000007 is probably prime
CO6	Parallel analysis	Three short texts, one per line	Three results with the word count of each text

7.3 Discussion

The tables below explain why each data structure and algorithm was chosen. The datasets are small samples written for testing, so the numbers show that the algorithms work correctly and can be checked by hand; they are not statements about real-world news. Sentiment uses a small word list, so sentences with sarcasm or mixed feelings may be classified wrongly.

Table 7.2: Phase 1 data structures

Feature	Data structure / algorithm	Time complexity
Keyword and hashtag counting	HashMap	O(1) average per update
Stop-word and sentiment look-up	HashSet	O(1) average
Top-K keywords / hashtags / trends	PriorityQueue (min-heap)	O(n log k)
Ranking posts by likes	Sorting (List.sort)	O(n log n)
Keyword search in posts	Linear search	O(n)
Search in sorted keyword list	Binary search	O(log n)

Table 7.3: Phase 2 query classes and algorithm families

Query class	Algorithm family	Algorithm used	Cost
Pattern search	String algorithms	KMP, Rabin–Karp	O(n + m)
Multi-keyword search	String algorithms	Aho–Corasick	O(n + m + z)
Fuzzy matching, text similarity	Dynamic programming	Edit distance, LCS	O(nm)
Capacity-constrained flow, cuts	Network flow	Edmonds–Karp	$O(V \cdot E^2)$
Choosing a small covering set	Approximation	Vertex-cover 2-approximation	2 × optimal
Primality of large numbers	Randomised algorithms	Miller–Rabin	Monte Carlo; error shrinks with each round

Query class	Algorithm family	Algorithm used	Cost
Many independent texts	Parallel algorithms	Parallel stream over documents	Work/span depends on hardware

Limitations. The application analyses the records stored in its own database and the text a user supplies. Live news and social-media collection needs an authorised provider or API and is not included. Sentiment and summaries are lightweight methods, and the assistant searches stored records instead of using a language model. The Miller–Rabin demonstration works on signed 64-bit integers, and the max-flow lab uses an adjacency matrix, which suits small graphs.

8. CONCLUSION & FUTURE SCOPE

Conclusion. The News & Social Media Analytics Engine successfully demonstrates how Data Structures and Algorithms can be applied to analyse and organise continuously growing textual data. By processing news and social-media content, the system identifies popular keywords, hashtags, trending topics and sentiment, giving a clear overview of the information in the dataset. The use of HashMap, Lists, searching, sorting and PriorityQueue makes the system efficient for storing, retrieving, counting and ranking data. In Phase 2 the engine became a full-stack web application with a React frontend, a Spring Boot backend and a MySQL database, secured with BCrypt hashing, sessions and role-based access. Selected algorithms from all six course outcomes (KMP, Rabin–Karp, Aho–Corasick, edit distance, LCS, Edmonds–Karp, vertex-cover approximation, Miller–Rabin and parallel analysis) are integrated and can be run interactively, so all the objectives of the project were met.

Future scope.

- **Near term:** replace the small word lists with a full sentiment lexicon, add stemming so that related words are counted together, and import larger CSV files.
- **Medium term:** use the citation_edges table for related-article graphs and bipartite matching, add time-window trend detection using a queue, add email alerts, and use suffix arrays for substring similarity.
- **Long term:** connect the engine to live news and social-media APIs with proper authorisation, scale it with a distributed framework such as Kafka and Spark, and add machine-learning models for topic detection and multilingual sentiment.

9. REFERENCES

1. Apache Storm – Distributed real-time computation system. https://storm.apache.org

2. Apache Kafka – Distributed event streaming platform. https://kafka.apache.org

3. Elasticsearch – Distributed search and analytics engine (inverted index). https://www.elastic.co/elasticsearch

4. NetworkX – Network analysis in Python. https://networkx.org

5. C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text," ICWSM 2014. https://github.com/cjhutto/vaderSentiment

6. The GDELT Project. https://www.gdeltproject.org

7. Google Trends. https://trends.google.com

8. Brandwatch – Social listening and analytics. https://www.brandwatch.com

9. Sprout Social – Social media analytics. https://sproutsocial.com

10. Oracle, Java Platform SE Documentation – Collections Framework (HashMap, ArrayList, PriorityQueue). https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/package-summary.html

11. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot

12. React Documentation. https://react.dev

13. Vite Documentation. https://vite.dev

14. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

15. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press (string matching, dynamic programming, maximum flow, NP-completeness and approximation).

16. J. Kleinberg and É. Tardos, Algorithm Design, Pearson (network flow, NP-completeness).

17. J. Erickson, Algorithms, free textbook. https://jeffe.cs.illinois.edu/teaching/algorithms/